SQL JOINs

Easy Step-by-Step Guide

Tank54 Group | Oracle HR Schema

What is a JOIN?

A JOIN puts together data from two tables. For example, the EMPLOYEES table has department_id, but not the department name. The DEPARTMENTS table has the department name. With a JOIN, we can see the employee name and the department name together in one result.

This guide explains INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN, and how to join 3 or more tables with simple examples.

The Two Tables We Will Use

For all examples, we use two tables: **EMPLOYEES** and **DEPARTMENTS**. They are connected by a common column: **DEPARTMENT_ID**. This column exists in both tables. We use this column to join the tables together.

EMPLOYEES Table (sample rows):

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	JOB_ID	SALARY	DEPARTMENT_ID
100	Steven	King	AD_PRES	24000	90
101	Neena	Kochhar	AD_VP	17000	90
102	Lex	De Haan	AD_VP	17000	90
107	Diana	Lorentz	IT_PROG	4200	60
178	Kimberely	Grant	SA_REP	7000	null

DEPARTMENTS Table (sample rows):

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID
10	Administration	1700
60	IT	1400
90	Executive	1700
270	Payroll	1700

Key Idea: EMPLOYEES.DEPARTMENT_ID = DEPARTMENTS.DEPARTMENT_ID
This is the **join condition**. It tells SQL: "match the employee's department ID with the department's ID, and put the data together".

JOIN Types - Overview

There are 4 main types of JOINs. Each type shows different rows in the result. The table below explains each type in simple words.

JOIN Type	What does it do?	Simple English
INNER JOIN	Shows ONLY matching rows from both tables.	Give me employees who have a department, and show me the department name.
LEFT JOIN	Shows ALL rows from the left table + matching rows from the right table.	Give me ALL employees. If they have a department, show it. If not, show null.
RIGHT JOIN	Shows ALL rows from the right table + matching rows from the left table.	Give me ALL departments. If they have employees, show them. If not, show null.
FULL OUTER JOIN	Shows ALL rows from both tables. Matching + non-matching.	Give me everything. All employees and all departments, matched or not.

Remember:
- "Left" table = the table you write first (after FROM)
- "Right" table = the table you write second (after JOIN)
- "Match" = the join condition (ON ... = ...)

1. INNER JOIN

Task:

Show each employee's first name, last name, and their department name. Show only employees who HAVE a department.

Step-by-Step:

Step 1: We need data from two tables: EMPLOYEES (for name) and DEPARTMENTS (for department name).

Step 2: The common column is DEPARTMENT_ID. EMPLOYEES.DEPARTMENT_ID must equal DEPARTMENTS.DEPARTMENT_ID.

Step 3: We use table aliases: "e" for EMPLOYEES and "d" for DEPARTMENTS. This makes the query shorter.

Step 4: INNER JOIN shows ONLY rows that match in both tables. Employee 178 (Kimberely) has null department_id, so she will NOT appear in the result. Department 270 (Payroll) has no employees, so it will NOT appear either.

SQL Query:

```
SELECT e.first_name,
     e.last_name,
     d.department_name
FROM employees e
INNER JOIN departments d
     ON e.department_id = d.department_id
ORDER BY e.last_name;
```

Result (sample):

First Name	Last Name	Department Name
Diana	Lorentz	IT
Lex	De Haan	Executive
Neena	Kochhar	Executive
Steven	King	Executive

Note: Employee Kimberely is NOT in the result (she has no department). Department Payroll is NOT in the result (it has no employees). This is how INNER JOIN works.

2. LEFT JOIN

Task:

Show ALL employees and their department name. If an employee has no department, show null.

Step-by-Step:

Step 1: EMPLOYEES is the left table (it comes after FROM). DEPARTMENTS is the right table (after JOIN).

Step 2: LEFT JOIN keeps ALL rows from the left table (EMPLOYEES). For matching rows, it shows the department name. For non-matching rows, it shows null.

Step 3: Employee 178 (Kimberely) has null department_id. With LEFT JOIN, she WILL appear in the result, but her department_name will be null.

Step 4: Department 270 (Payroll) still does NOT appear, because it is in the right table and has no matching employees.

SQL Query:

```
SELECT e.first_name,
     e.last_name,
     d.department_name
FROM employees e
LEFT JOIN departments d
     ON e.department_id = d.department_id
ORDER BY e.last_name;
```

Result (sample):

First Name	Last Name	Department Name
Diana	Lorentz	IT
Kimberely	Grant	null
Lex	De Haan	Executive
Neena	Kochhar	Executive
Steven	King	Executive

Note: Kimberely appears now (with null department)! Department Payroll is still missing (it is in the right table). The red row shows the difference from INNER JOIN.

3. RIGHT JOIN

Task:

Show ALL departments and their employees. If a department has no employees, show null for the employee.

Step-by-Step:

Step 1: EMPLOYEES is the left table. DEPARTMENTS is the right table. RIGHT JOIN keeps ALL rows from the right table (DEPARTMENTS).

Step 2: Department 270 (Payroll) has no employees. With RIGHT JOIN, it WILL appear in the result, but the employee columns will be null.

Step 3: Employee 178 (Kimberely) is still missing because she is in the left table and has no matching department.

SQL Query:

```
SELECT e.first_name,
     e.last_name,
     d.department_name
FROM employees e
RIGHT JOIN departments d
     ON e.department_id = d.department_id
ORDER BY d.department_name;
```

Result (sample):

First Name	Last Name	Department Name
Lex	De Haan	Executive
Neena	Kochhar	Executive
Steven	King	Executive
Diana	Lorentz	IT
null	null	Payroll

Note: Payroll appears now (with null employees)! Kimberely is still missing. RIGHT JOIN = opposite of LEFT JOIN.

4. FULL OUTER JOIN

Task:

Show ALL employees and ALL departments. Matched pairs show both names. Unmatched employees show null department. Unmatched departments show null employee.

Step-by-Step:

Step 1: FULL OUTER JOIN shows everything. All rows from both tables. Nothing is left out.

Step 2: Kimberely (no department) appears with null department_name.

Step 3: Payroll (no employees) appears with null employee names.

Step 4: All other matched rows appear normally, like in INNER JOIN.

SQL Query:

```
SELECT e.first_name,
     e.last_name,
     d.department_name
FROM employees e
FULL OUTER JOIN departments d
     ON e.department_id = d.department_id
ORDER BY d.department_name, e.last_name;
```

Result (sample):

First Name	Last Name	Department Name
Lex	De Haan	Executive
Neena	Kochhar	Executive
Steven	King	Executive
Diana	Lorentz	IT
null	null	Payroll
Kimberely	Grant	null

Note: EVERYTHING appears! Kimberely, Payroll, and all matched rows. FULL OUTER JOIN = INNER JOIN + LEFT JOIN unmatched + RIGHT JOIN unmatched.

5. Joining More Than 2 Tables

In real work, you often need data from 3 or more tables. For example, an employee works in a department, and that department is in a city. If we want to see the employee name, department name, AND city together, we need to join 3 tables: EMPLOYEES, DEPARTMENTS, and LOCATIONS.

How Do 3 Tables Connect?

Think of it like a chain. Each table links to the next one by a common column:

Table 1 connects to Table 2. Table 2 connects to Table 3. They form a chain. SQL follows this chain step by step.

LOCATIONS Table (sample rows):

LOCATION_ID	CITY	COUNTRY_ID
1400	South San Francisco	US
1700	Seattle	US

Manual Process (How You Would Do It by Hand)

Imagine you have 3 papers on your desk. Paper 1 = EMPLOYEES. Paper 2 = DEPARTMENTS. Paper 3 = LOCATIONS. You want to make one list with employee name, department name, and city.

Step 1: Take EMPLOYEES paper. Pick the first row: Steven, department_id = 90.

Step 2: Go to DEPARTMENTS paper. Find the row where department_id = 90. You find: Executive, location_id = 1700.

Step 3: Now go to LOCATIONS paper. Find the row where location_id = 1700. You find: Seattle.

Step 4: Write down: Steven | Executive | Seattle. This is the first row of your result.

Step 5: Go back to EMPLOYEES paper. Pick the next row: Neena, department_id = 90. Repeat steps 2-4.

Step 6: Do the same for every employee row. For employees with null department_id (like Kimberely), you cannot find a match in DEPARTMENTS, so the result shows null for department and city.

> **Key Idea:** You always go table by table, following the chain. First match Table 1 with Table 2, then use that result to match with Table 3. Each new table adds one more JOIN to your SQL.

SQL Code (How Each Manual Step Translates to SQL)

Manual Step 1-2 (Pick employee, find department): This is the first JOIN.

Manual Step 3 (Find city from location): This is the second JOIN.

SQL Query:

```
SELECT e.first_name,
     e.last_name,
     d.department_name,
     l.city
FROM employees e
INNER JOIN departments d
     ON e.department_id = d.department_id
INNER JOIN locations l
     ON d.location_id = l.location_id
ORDER BY e.last_name;
```

> **Manual to SQL Mapping:**
> - "Start from EMPLOYEES" = FROM employees e
> - "Find matching department" = INNER JOIN departments d ON e.department_id = d.department_id
> - "Find matching city" = INNER JOIN locations l ON d.location_id = l.location_id
> - "Write down these columns" = SELECT e.first_name, e.last_name, d.department_name, l.city

Result:

First Name	Last Name	Department Name	City
Diana	Lorentz	IT	South San Francisco

Lex	De Haan	Executive	Seattle
Neena	Kochhar	Executive	Seattle
Steven	King	Executive	Seattle

Note: We now have data from 3 tables in one result! Kimberely is not in the result because we used INNER JOIN. If we used LEFT JOIN for the first JOIN, she would appear with null department and null city.

General Rule for Multiple Tables

When you join N tables, you need N-1 JOIN lines in your SQL. For example:

Number of Tables	Number of JOIN Lines	Example
2 tables	1 JOIN	employees JOIN departments
3 tables	2 JOINs	employees JOIN departments JOIN locations
4 tables	3 JOINs	employees JOIN departments JOIN locations JOIN countries
5 tables	4 JOINs	employees JOIN departments JOIN locations JOIN countries JOIN regions

Important: Each JOIN must have an ON condition. The ON condition tells SQL how to connect this table to the previous one. Without ON, SQL does not know which rows match.

Remember the Pattern:
FROM table1 a
JOIN table2 b ON a.common_column = b.common_column
JOIN table3 c ON b.other_column = c.other_column
JOIN table4 d ON c.next_column = d.next_column

Each new JOIN uses a column from the PREVIOUS table to connect to the NEW table.

Quick Summary

JOIN Type	Kimberely (no dept)	Payroll dept (no employees)	Matched rows (e.g. Steven)
INNER JOIN	NO	NO	YES
LEFT JOIN	YES	NO	YES
RIGHT JOIN	NO	YES	YES
FULL OUTER JOIN	YES	YES	YES

When to Use Which JOIN?

Situation	Use This JOIN
I only want employees who have a department.	INNER JOIN
I want ALL employees, even those with no department.	LEFT JOIN
I want ALL departments, even those with no employees.	RIGHT JOIN
I want everything from both tables.	FULL OUTER JOIN

Useful Tip - Table Aliases:
Always use short aliases for table names in JOINs:
- employees e (short and easy)
- departments d

Then use the alias before column names:
- e.first_name (from employees)
- d.department_name (from departments)

If two tables have a column with the same name (like department_id), you MUST use the alias: e.department_id, d.department_id.